

Dokumentáció: Végleges

Készítette: Pallang Hunor

Tárgy: Programozás alapjai 2.

Feladat: BME Étterem Kezelő rendszer

Fájl: NHF 4 – Végleges Változat

1. Pontosított és Kibővített Feladatspecifikáció

1.1. A szoftver célja

A jelen **nagy házi feladat** célja egy étterem napi működését támogató digitális adminisztrációs rendszer kidolgozása konzolos felületen. A szoftver lehetővé teszi az étterem asztalainak, az étlap különböző típusú tételeinek és a vendégek rendeléseinek nyilvántartását. A program nem korlátozza a tárolható adatok mennyiségét, azokat a rendelkezésre álló memória erejéig dinamikusan kezeli.

1.2. Funkcionális követelmények és felhasználói esetek

1.2.1. Asztalmenedzsment és Kapacitáskezelés

- **Új asztal rögzítése:** A felhasználó új asztalt vezethet be a rendszerbe egyedi azonosító (ID), maximális férőhely (minimum 2 fő), rövid szöveges leírás (pl. „Ablak melletti box”), valamint az étterem diszkrét koordináta-rendszerében elfoglalt X és Y pozíció megadásával.
- **Asztal adatainak módosítása:** Lehetőség van a rögzített asztalok attribútumainak utólagos korrekciójára.
- **Asztal eltávolítása:** Egy asztal fizikai törlése csak akkor engedélyezett, ha az asztal szabad, és nem kapcsolódik hozzá aktív rendelési életciklus.
- **Fizetés és Lezárás (Számlázás):** A vendégek távozásakor a program összesíti a fogyasztási tételeket, kiszámítja a végösszeget, megjeleníti azt, formázott nyugtát exportál a külső tárolóba, majd az asztalt felszabadítja.

1.2.2. Heterogén Étlapkezelés

A kínálat nyilvántartása polimorf módon, egy közös absztrakt bázisosztályon keresztül történik:

- **Ételek (Food):** Rendelkeznek azonosítóval, névvel, egységgel, elérhetőségi státusszal (készleten van-e) és allergén információkkal.
- **Italok (Drink):** Rendelkeznek azonosítóval, névvel, egységgel, elérhetőséggel, valamint specifikus extra mezőkkel: űrtartalom (literben) és alkoholtartalom (logikai jelző).

A felhasználó az étlap elemeit interaktívan felveheti, módosíthatja (a Visitor minta segítségével `dynamic_cast` nélkül), törölheti, vagy kilistázhhatja.

1.2.3. Rendeléskezelés és Életciklus

- **Rendelés nyitása:** Szabad asztalhoz tétel rendelhető az étlapról a darabszám megadásával. Az első tétel hozzáadásakor az asztal státusza „foglalt”-ra változik.
- **Rendelés módosítása:** A felvett rendelések darabszáma módosítható. Amennyiben egy tétel mennyisége 0-ra csökken, a program azt megsemmisíti az aktív listából.
- **Lekérdezés:** Asztalonként megtekinthető az eddigi fogyasztás tételnevekkel, darabszámokkal, részösszegekkel és az aktuális végösszeggel.

1.2.4. Dinamikus Foglaltsági Térkép

A szoftver grafikus hatású, kétdimenziós karakteres térképet rajzol az étterem aktuális elrendezéséről a koordináták alapján. A szabad asztalok zöld, a foglalt asztalok piros színű egyedi azonosítós blokként jelennek meg (pl. [01]).

1.3. Bemeneti és kimeneti formátumok (Adatintegritás)

A program az adatperzisztenciát három szöveges adatfájlon keresztül biztosítja. Induláskor beolvassa, leálláskor felülírja őket.

- tables.txt: ID;férőhely;leírás;X;Y;foglaltság
 - *Példa:* 1;4;Ablak melletti box;5;4;1
- menu.txt: Típus(E/I);ID;Név;Ár;Elérhetőség;[Allergén VAGY Űrtartalom;Alkoholtartalom]
 - *Példa:* E;1;Margherita Pizza;2490;1;Glutén, Tej
- orders.txt: AsztalID;TételID;Darabszám (Minden egyes aktív rendelési tétel külön sorban szerepel)
 - *Példa:* 1;1;2

1.4. Generált kimenet (Számlázás)

Asztal lezárásakor a szoftver a receipts/ alkönyvtárba generál egy számlafájlt: receipt_YYYY_MM_DD_ID.txt. A fájl nyugtaformátumban tartalmazza a tételek részletezését és a végösszeget. A számlák egyedi sorszámozását a receipt_counter.txt segéd fájl kezeli, amely az adott napon kiállított számlák sorszámát tárolja.

- **Sorszámozási algoritmus:**
 1. A program lekéri a mai dátumot, és összeveti a receipt_counter.txt fájlban tárolt dátummal.
 2. Amennyiben a dátum egyezik, a tárolt sorszámot megnöveli eggyel; ellenkező esetben (vagy ha a fájl nem létezik) a számláló 1-ről indul.
 3. Az új állapot azonnal mentésre kerül a segéd fájlba, biztosítva a szekvenciális folytonosságot.

2. Megoldási Vázlat és Tervezői Döntések

2.1. Architektúra áttekintés

A szoftver szigorúan követi az **Objektumorientált Tervezési Elveket** és a **Clean Code** irányelveit (MVC elv). A rendszer magját a Modell alkotja (Restaurant, Table, Order), míg a teljes felhasználói interakciót a UI osztály vezérli. Ezzel a modell osztályai teljesen függetlenül egységtesztelhetővé váltak.

2.2. Alkalmazott tervezési minták és idiómák

- **Facade (Homlokzat) minta:** A Restaurant osztály egységes interfészt biztosít a külvilág felé az étterem alrendszereihez.

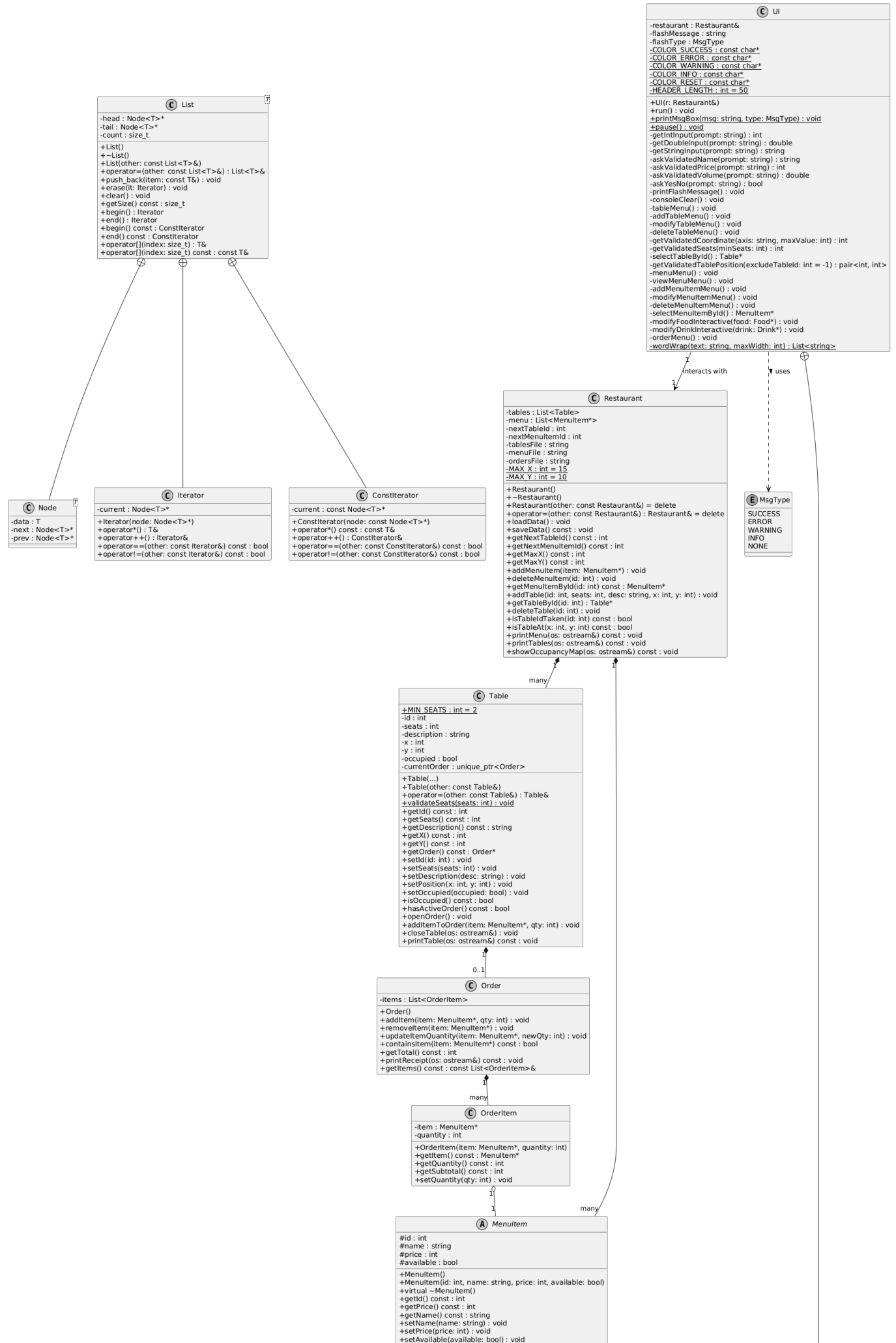
- **Prototípus (Prototype) minta:** Az étlap elemeinek másolása a virtuális clone() metódussal történik a "slicing" hiba elkerülése végett.
- **Visitor (Látogató) minta a dynamic_cast elkerülésére:** Az étlap elem interaktív módosításakor a UI meghívja az item->accept(visitor) metódust. A Double Dispatch révén a vezérlés a pontos leszármazott osztályba ágyazott accept-re ugrik, elkerülve a kasztolásokat.
- **Iterator és Constliterator idióma:** A saját láncolt lista (List) megvalósítja a standard C++ iterátor design, így a kód képes a tartományalapú for-ciklusok használatára.

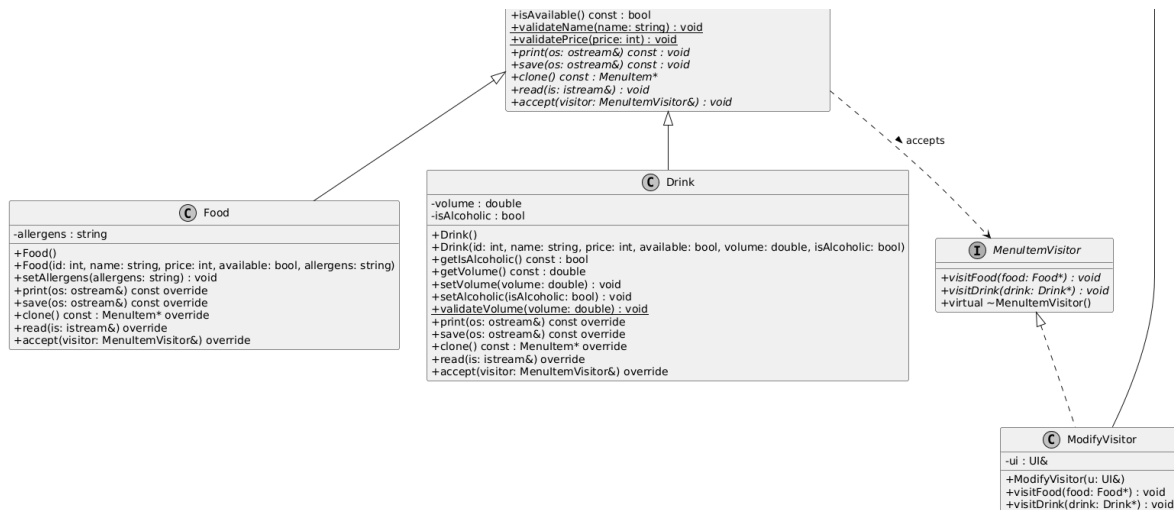
2.3. Memóriakezelési stratégia

A szoftver tiltott STL konténerek helyett saját, duplán láncolt listát (List) használ. A memóriaszivárgást a **Hármas Szabály (Rule of Three)** betartása garantálja. Az asztalokhoz tartozó rendelések kezelésénél **std::unique_ptr** okospointert alkalmaztam, amely automatikusan felszabadítja az Order objektumot, megakadályozva a dangling pointer kialakulását.

3. Osztálydiagram és a Komponensek Részletes Leírása

3.1. Rendszerterv: Osztálydiagram (UML)





3.2. Osztályok és Metódusok Részletes Specifikációja

- **List:** Generikus láncolt lista. Főbb metódusai: `push_back(item)`, `erase(iterator)`, `clear()`, `operator[]`. A memória szivárgásmentességéről a destruktor és a másoló konstruktorok gondoskodnak. Tartalmazza az `Iterator` és `ConstIterator` belső osztályokat.
- **Restaurant:** A központi üzleti modell (Facade). Kezeli az I/O perzisztenciát (`loadData`, `saveData`). A `deleteMenuItem(id)` végrehajtja a biztonsági ellenőrzést: ha a tétel aktív rendelésben szerepel, `std::logic_error` kivételt dob.
- **Table:** Asztal entitás. Az `openOrder()` új `Order` objektumot példányosít a heapen. A `closeTable(ostream&)` kinyomtatja a nyugtát és a `currentOrder.reset()` hívással megsemmisíti a rendelőt.
- **Order & OrderItem:** Az `Order::updateItemQuantity` metódus automatikusan törli a tételt a listából, ha a megadott darabszám 0. A `getTotal()` kiszámítja a végösszeget.
- **MenuItem, Food, Drink:** Polimorf étlap hierarchia. A `clone()` garantálja a helyes másolást. Az `accept(MenuItemVisitor&)` metódus a típusfüggő (dynamic_cast nélküli) hívások belépési pontja.
- **UI & ModifyVisitor:** A UI magába zárja az interakciókat. Validált bemeneti függvényeket (`getIntInput`, stb.) használ. A `ModifyVisitor` valósítja meg a Látogató mintát a típusbiztos szerkesztéshez.

4. Tesztelési Dokumentáció és Verifikáció

4.1. Automata Egységtesztek (qtest_lite)

Az üzleti logika automatizált ellenőrzését a `main.cpp` alatt elhelyezett, `gtest_lite.h` alapú tesztblokk végzi. A tesztek lefedik az összes kritikus elágazást:

1. **ListTest.InitAndPushBack:** Üres lista inicialása, elemek beszúrása, iterálás tesztelése.
2. **ListTest.OutOfRangeThrow:** Túlindexelés esetén `std::out_of_range` kivétel dobásának ellenőrzése.
3. **ListTest.DeepCopyAndAssignment:** A **Hármas Szabály** működésének igazolása mély másolással (Deep Copy).
4. **ListTest.EraseAndClear:** Elemek fizikai törlésének és a teljes memória kiürítésének (clear) tesztje.
5. **OrderTest.AddAndCalculateTotal:** A `getTotal()` metódus helyes matematikai összegzésének ellenőrzése.
6. **OrderTest.UpdateQuantityAndRemove:** Tétel 0 darabra állításának és ezáltal automatikus törlésének ellenőrzése.

7. **RestaurantTest.TableCollisionHandling:** Ütközésvédelem tesztje (foglalt ID vagy koordináta esetén std::invalid_argument dobása).
8. **RestaurantTest.MenuManagement:** Heterogén kollekció kezelése és ID alapján történő keresés (nullptr visszatérés tesztje).
9. **RestaurantTest.DeleteNonExistentItem:** Nem létező tétel törlésének hiba-ellenőrzése.
10. **RestaurantTest.DeleteItemInActiveOrder: Biztonsági Főteszt (Dangling Pointer védelem).** Aktív rendelésben szereplő étel törlésének megtagadása (std::logic_error kivétellel).
11. **MenuItemTest.FoodAndDrinkProperties:** A leszármazott osztályok (Food, Drink) polimorf viselkedésének tesztelése.
12. **TableTest.OrderManagement:** Az asztal állapotgépének verifikálása (rendelés nyitáskor occupied státusz ellenőrzése).
13. **FoodTest.AllergenHandling:** Az allergénkezelés és a név/ár lekérdezés helyességének ellenőrzése.
14. **FoodTest.FoodProperties:** Az Food objektum alapadatainak és elérhetőségi állapotának tesztelése.
15. **DrinkTest.VolumeValidation:** Az ital űrtartalom-ellenőrzésének és az alkoholtartalom lekérdezésének tesztje.
16. **DrinkTest.AlcoholicFlag:** Az alkoholtartalom kapcsoló módosíthatóságának vizsgálata.
17. **OrderItemTest.Construction:** Az OrderItem konstruktorának, mennyiségének és részösszegének ellenőrzése.
18. **OrderItemTest.NullptrThrow:** Null pointer átadása esetén kivételdobás tesztje.
19. **OrderItemTest.UpdateQuantity:** A darabszám módosításának és a részösszeg újraszámításának ellenőrzése.
20. **TableTest.CopyConstructor:** A Table másoló konstruktorának helyes működése.
21. **TableTest.AssignmentOperator:** A Table értékadó operátorának helyes működése.
22. **TableTest.ValidateSeats:** A férőhely-validáció tesztelése a konstruktornál.
23. **TableTest.SetSeatsValidation:** A setSeats validációjának ellenőrzése.
24. **TableTest.SetDescription:** A leírás módosíthatóságának vizsgálata.
25. **TableTest.PositionHandling:** Az asztal koordinátakezelésének tesztje.
26. **OrderTest.AddSameItemMultipleTimes:** Azonos tétel többszöri hozzáadásakor a mennyiség helyes növelésének ellenőrzése.
27. **OrderTest.ContainsItem:** A containsItem lekérdezés működésének tesztje.
28. **OrderTest.RemoveItem:** Egy tétel eltávolításának és a végösszeg újraszámításának ellenőrzése.
29. **OrderTest.AddNullItem:** Null pointer hozzáadásának hibamentes kezelése.
30. **OrderTest.AddNegativeQuantity:** Negatív mennyiség kezelésének hibamentes elutasítása.
31. **ListTest.IteratorOperations:** Iterátorhasználat és összegzés tesztelése.
32. **ListTest.EmptyListIterator:** Üres lista iterálásának ellenőrzése.
33. **ListTest.SingleElementIterator:** Egyetlen elem iterátorral történő bejárásának tesztje.
34. **RestaurantTest.MapDimensions:** Az étterem pályaméretének ellenőrzése.
35. **RestaurantTest.MultipleTablesHandling:** Több asztal együttes kezelésének és lekérdezésének tesztje.
36. **RestaurantTest.MultipleMenuItemsHandling:** Több menüelem nyilvántartásának és ID-alapú lekérdezésének tesztje.
37. **TableTest.OccupiedWithActiveOrder:** Aktív rendelés mellett az elfoglaltsági állapot módosításának tilalma.
38. **TableTest.CloseTableOperations:** Az asztal lezárásának és a rendelés megszüntetésének tesztje.
39. **OrderTest.ComplexTotal:** Több különböző tételből számolt végösszeg ellenőrzése.
40. **ListTest.StringList:** Sztringeket tároló lista helyes működésének tesztje.

4.2. Memóriaszivárgás-mentesség ellenőrzése (Memtrace)

A memóriabiztonság igazolása a BME IIT által biztosított **memtrace** könyvtárral történt.

- A fordítás a -DMEMTRACE makróval történt, amely regisztrálta a new és delete hívásokat.
- Az etterem_tesztek (mind a 40 egységteszt) futtatása után a memtrace automatikusan kiértékelte az allokációs táblát.
- A rendszer **0 bajtshivárgást** igazolt, ami megerősíti, hogy a List destruktora és az std::unique_ptr okosponterek hibátlanul takarítják fel a memóriát.